Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

100 In-Depth Questions with Code Examples

> **Prepared for:** SDE2 role interviews | **Stack:** Go (Golang) | **Level:** Mid-level (2–4 years)

Table of Contents

Goroutines & Concurrency

Q1. What is a goroutine and how is it different from an OS thread?

Pattern: Fundamentals | Difficulty: Easy

A goroutine is a lightweight function execution managed by the Go runtime scheduler, not the OS. OS threads are typically 1–8 MB stack, fixed. Goroutines start at 2 KB and grow dynamically. The Go runtime multiplexes M goroutines onto N OS threads (M:N scheduling).

```
go func() {
    fmt.Println("runs concurrently")
}()
```

> ***Interview tip:*** *"Goroutines are cheap because the stack starts small and grows on demand. You can run millions; you cannot do the same with OS threads."*

Q2. What is a goroutine leak and how do you detect and fix it?

Pattern: Leak detection | Difficulty: Medium

A goroutine leak occurs when a goroutine is started but never terminates — usually blocked on a channel that never receives/sends. Detect with `runtime.NumGoroutine()` or pprof goroutine profile.

```
// Leak — blocks forever
func leak() {
    ch := make(chan int)
    go func() {
        v := <-ch // never unblocks
        fmt.Println(v)
    }()
}

// Fix — use context cancellation
func noLeak(ctx context.Context) {
    ch := make(chan int)
    go func() {
        select {
        case v := <-ch:
            fmt.Println(v)
        case <-ctx.Done():
            return
```

```
        }
    }()
}
```

> ***Interview tip:*** *"Always ask: who closes this channel / who cancels this context?"*

Q3. Explain the Go scheduler — what is GMP?

Pattern: Runtime internals | **Difficulty: Hard**

GMP = Goroutine, Machine (OS thread), Processor (logical CPU slot). Each P has a local run queue. An M runs goroutines by picking from a P's queue. Work stealing: idle Ps steal from busy Ps.

```
runtime.GOMAXPROCS(4)              // set number of Ps
fmt.Println(runtime.NumGoroutine()) // count goroutines
```

> ***Interview tip:*** *Draw the G→P→M triangle on a whiteboard. This separates mid-level from senior candidates at Go-heavy companies.*

Q4. What is GOMAXPROCS and what is its default value?

Pattern: Runtime configuration | **Difficulty: Easy**

`GOMAXPROCS` controls the number of OS threads that can execute Go code simultaneously. Default since Go 1.5 is the number of CPU cores available (`runtime.NumCPU()`).

```
prev := runtime.GOMAXPROCS(8) // set to 8, returns old value
fmt.Println(runtime.GOMAXPROCS(0)) // read current value
```

> ***Interview tip:*** *Setting it to 1 makes goroutines cooperative (single thread). Useful for debugging race conditions.*

Q5. What is the difference between concurrency and parallelism in Go?

Pattern: Conceptual | **Difficulty: Easy**

Concurrency is about *structure* — composing independent tasks. Parallelism is about *execution* — running tasks simultaneously. Go makes concurrency easy via goroutines; parallelism depends on `GOMAXPROCS` and CPU cores.

```
// Concurrent — two goroutines, may run on same thread
go doA()
go doB()

// Parallel — runs on multiple CPUs (needs GOMAXPROCS > 1)
runtime.GOMAXPROCS(runtime.NumCPU())
go doA()
go doB()
```

> ***Interview tip:*** *Rob Pike's quote: "Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once."*

Q6. How do you implement a worker pool in Go?

Pattern: Worker pool | **Difficulty: Medium**

Fixed number of goroutines pull work from a shared jobs channel. Prevents spawning too many goroutines under load.

```
func workerPool(numWorkers int, jobs <-chan int, results chan<- int) {
    var wg sync.WaitGroup
    for i := 0; i < numWorkers; i++ {
```

```
        wg.Add(1)
        go func() {
            defer wg.Done()
            for job := range jobs {
                results <- job * 2 // process work
            }
        }()
    }
    go func() {
        wg.Wait()
        close(results)
    }()
}

// Usage
jobs := make(chan int, 100)
results := make(chan int, 100)
workerPool(5, jobs, results)
for i := 0; i < 20; i++ { jobs <- i }
close(jobs)
for r := range results { fmt.Println(r) }
```

> ***Interview tip:*** *Worker pool is a mandatory LLD pattern for Go SDE2. Be ready to code it from scratch in 10 minutes.*

Q7. What is sync.WaitGroup and how does it work?

Pattern: Synchronisation | **Difficulty: Easy**

`sync.WaitGroup` waits for a collection of goroutines to finish. `Add(n)` increments the counter. `Done()` decrements. `Wait()` blocks until counter reaches 0.

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        fmt.Printf("worker %d done\n", id)
    }(i)
}
wg.Wait()
fmt.Println("all workers done")
```

> ***Interview tip:*** *Always call `wg.Add()` before launching the goroutine, not inside it. The goroutine might be scheduled after `Wait()` is called.*

Q8. What is a data race in Go and how do you detect and prevent it?

Pattern: Race detection | **Difficulty: Hard**

A data race occurs when two goroutines access the same memory concurrently and at least one writes, without synchronisation.

```
// Race condition — WRONG
var counter int
go func() { counter++ }()
go func() { counter++ }()

// Fix 1: Mutex
var mu sync.Mutex
```

```
go func() { mu.Lock(); counter++; mu.Unlock() }()

// Fix 2: atomic
var atomicCounter int64
go func() { atomic.AddInt64(&atomicCounter, 1) }()
```

```
# Detect races at test time
go test -race ./...
go run -race main.go
```

> ***Interview tip:*** *Always run tests with `-race`. The race detector has near-zero false positives.*

Q9. When do you use sync.Mutex vs sync.RWMutex?

Pattern: Locking strategy

Difficulty: Medium

`sync.Mutex` — exclusive lock. Only one goroutine at a time (read or write). `sync.RWMutex` — multiple concurrent readers OR one exclusive writer. Use `RWMutex` when reads vastly outnumber writes.

```
type SafeMap struct {
    mu sync.RWMutex
    m  map[string]int
}

func (s *SafeMap) Get(key string) int {
    s.mu.RLock()         // multiple readers can hold this simultaneously
    defer s.mu.RUnlock()
    return s.m[key]
}

func (s *SafeMap) Set(key string, val int) {
    s.mu.Lock()          // exclusive — blocks all readers and writers
    defer s.mu.Unlock()
    s.m[key] = val
}
```

> ***Interview tip:*** *RWMutex has overhead; for write-heavy workloads a plain Mutex can be faster. Always benchmark.*

Q10. What is sync.Once and when should you use it?

Pattern: Singleton / lazy init

Difficulty: Medium

`sync.Once` ensures a function runs exactly once regardless of how many goroutines call it. The canonical use: lazy singleton initialisation.

```
var (
    instance *Database
    once     sync.Once
)

func GetDB() *Database {
    once.Do(func() {
        instance = &Database{
            conn: mustConnect(os.Getenv("DSN")),
        }
    })
    return instance
}
```

Interview tip: *"Why not a mutex with nil check?" — sync.Once has no risk of forgetting unlock and correctly handles concurrent first-calls without double-checked locking bugs.*

Q11. How do you implement a fan-out / fan-in pattern?

Pattern: Pipeline

Difficulty: Hard

Fan-out: distribute work from one channel to multiple goroutines. Fan-in: merge multiple channels into one result channel.

```
func fanOut(in <-chan int, workers int) []<-chan int {
    outs := make([]<-chan int, workers)
    for i := 0; i < workers; i++ {
        out := make(chan int)
        outs[i] = out
        go func(o chan int) {
            for v := range in { o <- process(v) }
            close(o)
        }(out)
    }
    return outs
}

func fanIn(cs ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    for _, c := range cs {
        wg.Add(1)
        go func(ch <-chan int) {
            defer wg.Done()
            for v := range ch { out <- v }
        }(c)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

Interview tip: *The `WaitGroup` pattern to close the merged channel after all producers finish is the key detail interviewers look for.*

Q12. What is a semaphore in Go and how do you implement one?

Pattern: Rate / resource limiting

Difficulty: Medium

A semaphore limits concurrent access to a resource. In Go, a buffered channel with capacity N acts as a semaphore — send to acquire, receive to release.

```
type Semaphore chan struct{}

func NewSemaphore(n int) Semaphore {
    return make(Semaphore, n)
}

func (s Semaphore) Acquire() { s <- struct{}{} }
func (s Semaphore) Release() { <-s }

// Usage: limit to 3 concurrent DB queries
sem := NewSemaphore(3)
```

```
for _, item := range items {
    sem.Acquire()
    go func(i Item) {
        defer sem.Release()
        queryDB(i)
    }(item)
}
```

> ***Interview tip:*** *`golang.org/x/sync/semaphore` provides a weighted semaphore with context support. Mention it as a production alternative.*

Q13. How does Go handle goroutine preemption?

Pattern: Scheduler internals | **Difficulty: Hard**

Before Go 1.14, goroutines could only be preempted at function call boundaries (cooperative). Since Go 1.14, asynchronous preemption allows any goroutine to be suspended at safe points via signals (SIGURG on Unix), preventing CPU hogging in tight loops.

```
// Pre-1.14: this loop could hog a CPU forever
// Post-1.14: runtime sends SIGURG to preempt it
go func() {
    for {
        // tight loop with no function calls
        x++
    }
}()
```

> ***Interview tip:*** *This explains why `runtime.Gosched()` is rarely needed in modern Go — the scheduler handles it automatically.*

Q14. What is the difference between a goroutine and a coroutine?

Pattern: Conceptual | **Difficulty: Medium**

Coroutines are cooperatively scheduled — they yield explicitly. Goroutines are preemptively scheduled by the Go runtime. Goroutines can run in parallel across threads; coroutines are single-threaded.

```
// Goroutine: scheduled by runtime, may run in parallel
go task()

// Manual yield (rarely needed, hint to scheduler)
runtime.Gosched()
```

> ***Interview tip:*** *"Go goroutines are M:N — multiplexed onto OS threads. Coroutines (like Python's asyncio) are 1:1 with their event loop thread."*

Q15. How do you gracefully shut down multiple goroutines?

Pattern: Graceful shutdown | **Difficulty: Hard**

Use a `context.Context` for cancellation and a `sync.WaitGroup` to wait for all goroutines to finish before exiting.

```
func main() {
    ctx, cancel := context.WithCancel(context.Background())
    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {
        wg.Add(1)
```

```
        go func(id int) {
            defer wg.Done()
            for {
                select {
                case <-ctx.Done():
                    fmt.Printf("worker %d shutting down\n", id)
                    return
                default:
                    doWork()
                }
            }
        }(i)
    }

    // Handle OS signals
    sigCh := make(chan os.Signal, 1)
    signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)
    <-sigCh

    cancel()  // signal all goroutines to stop
    wg.Wait() // wait for all to finish
    fmt.Println("shutdown complete")
}
```

> ***Interview tip:*** *This is the production pattern for HTTP servers, background workers, and queue consumers. Know it cold.*

Channels & Select

Q16. What is the difference between buffered and unbuffered channels?

Pattern: Channel fundamentals

Difficulty: Easy

Unbuffered (`make(chan T)`) — sender blocks until receiver is ready. Both must be ready simultaneously — a synchronisation point. Buffered (`make(chan T, n)`) — sender blocks only when buffer full; receiver blocks only when buffer empty.

```
// Unbuffered — synchronisation
ch := make(chan int)

// Buffered — decoupling
bch := make(chan int, 3)
bch <- 1; bch <- 2; bch <- 3
// bch <- 4 would block here
```

> ***Interview tip:*** *"Unbuffered channels guarantee synchronisation. Buffered channels decouple producers from consumers."*

Q17. What happens when you send to a closed channel?

Pattern: Channel safety

Difficulty: Medium

Sending to a closed channel panics. Receiving from a closed channel returns the zero value immediately (and `false` as the second return). Only the sender should close a channel.

```
ch := make(chan int, 1)
ch <- 42
close(ch)

v, ok := <-ch  // v=42, ok=true  (buffered value drained)
v, ok = <-ch   // v=0,  ok=false (channel closed, zero value)

// ch <- 1 → PANIC: send on closed channel
```

Interview tip: *"Close is a broadcast — all receivers see it. Use it to signal 'no more values' to multiple goroutines."*

Q18. How does the select statement work? What happens with multiple ready cases?

Pattern: select semantics

Difficulty: Medium

`select` waits on multiple channel operations. When multiple cases are ready simultaneously, Go picks one uniformly at random. A `default` case makes it non-blocking.

```
select {
case msg := <-ch1:
    fmt.Println("ch1:", msg)
case msg := <-ch2:
    fmt.Println("ch2:", msg)
case ch3 <- "hello":
    fmt.Println("sent to ch3")
case <-time.After(1 * time.Second):
    fmt.Println("timeout")
default:
    fmt.Println("no channel ready — non-blocking")
}
```

Interview tip: *The random selection is intentional to prevent starvation. Ask "what if both channels fire at once?" to show you know this.*

Q19. How do you implement a timeout using channels?

Pattern: Timeout pattern

Difficulty: Easy

Use `time.After` or `context.WithTimeout` in a `select` to implement a timeout.

```
func fetchWithTimeout(url string) (string, error) {
    result := make(chan string, 1)
    go func() {
        result <- fetch(url) // potentially slow
    }()

    select {
    case r := <-result:
        return r, nil
    case <-time.After(3 * time.Second):
        return "", errors.New("request timed out")
    }
}
```

Interview tip: *Prefer `context.WithTimeout` over `time.After` in production — it propagates cancellation down the call stack.*

Q20. What is a done channel pattern?

Pattern: Cancellation broadcast

Difficulty: Medium

A done channel signals goroutines to stop. Closing it broadcasts to all listeners simultaneously — the preferred way to cancel multiple goroutines before context was introduced.

```
func generator(done <-chan struct{}, nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            select {
            case out <- n:
            case <-done: // stop if cancelled
                return
            }
        }
    }()
    return out
}

done := make(chan struct{})
ch := generator(done, 1, 2, 3, 4, 5)
fmt.Println(<-ch) // 1
close(done)       // cancel all goroutines listening on done
```

> ***Interview tip:*** *Today use `context.Context` instead of done channels. But understanding done channels shows you know the evolution of Go patterns.*

Q21. How do you implement a pipeline in Go?

Pattern: Pipeline

Difficulty: Medium

A pipeline is a series of stages connected by channels. Each stage takes input from one channel, processes it, and sends output to another channel.

```
func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        for _, n := range nums { out <- n }
        close(out)
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in { out <- n * n }
        close(out)
    }()
    return out
}

func main() {
    // Stage 1 → Stage 2 → consume
    for v := range square(square(generate(2, 3, 4))) {
        fmt.Println(v) // 16, 81, 256
```

```
    }
}
```

> ***Interview tip:*** *Pipelines are composable. Each stage is independent and can be parallelised. Closing channels propagates the "done" signal downstream.*

Q22. What is channel direction and why is it important?

Pattern: Type safety | **Difficulty: Medium**

Channel directions (`chan<- T` send-only, `<-chan T` receive-only) restrict how a channel is used. They provide compile-time guarantees, preventing accidental sends or receives.

```
func producer(out chan<- int) { // can only send
    for i := 0; i < 5; i++ { out <- i }
    close(out)
}

func consumer(in <-chan int) { // can only receive
    for v := range in { fmt.Println(v) }
}

func main() {
    ch := make(chan int, 5) // bidirectional
    go producer(ch)         // implicitly converts to chan<- int
    consumer(ch)            // implicitly converts to <-chan int
}
```

> ***Interview tip:*** *"Directional channels are documentation and safety. They tell the reader who owns sending and who owns receiving."*

Q23. How do you range over a channel?

Pattern: Channel iteration | **Difficulty: Easy**

`for range` on a channel receives values until the channel is closed. Without `close()`, the loop blocks forever.

```
ch := make(chan int, 5)
for i := 0; i < 5; i++ { ch <- i }
close(ch) // MUST close, otherwise range blocks forever

for v := range ch {
    fmt.Println(v) // 0, 1, 2, 3, 4
}
```

> ***Interview tip:*** *"Who is responsible for closing?" — always the sender. Closing from the receiver side risks panic on subsequent sends.*

Q24. How do you merge multiple channels in Go?

Pattern: Fan-in / merge | **Difficulty: Medium**

Use a goroutine per input channel, all sending to a shared output channel. Use `sync.WaitGroup` to close the output when all inputs are drained.

```
func merge(channels ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    for _, ch := range channels {
        wg.Add(1)
```

```
        go func(c <-chan int) {
            defer wg.Done()
            for v := range c { out <- v }
        }(ch)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

> ***Interview tip:*** *This is the fan-in half of the fan-out/fan-in pattern. Know both halves.*

Q25. What happens if you receive from a nil channel?

Pattern: Channel nil semantics | **Difficulty: Medium**

Receiving from a `nil` channel blocks forever. Sending to a `nil` channel blocks forever. Closing a `nil` channel panics. Nil channels in `select` are skipped — useful for dynamic disabling.

```
var ch chan int // nil

// These block forever:
// v := <-ch
// ch <- 1

// close(ch) → PANIC

// In select, nil channel case is never selected:
select {
case v := <-ch: // ch is nil → this case is never ready
    fmt.Println(v)
case <-time.After(1 * time.Second):
    fmt.Println("timeout") // this fires
}
```

> ***Interview tip:*** *Nil channel in select is a real pattern — disable a case dynamically by setting the channel variable to nil.*

Q26. How do you implement a ticker/interval in Go?

Pattern: time.Ticker | **Difficulty: Easy**

`time.Ticker` sends on a channel at regular intervals. Always stop it to avoid goroutine leak. `time.After` is one-shot.

```
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop() // IMPORTANT: prevent goroutine leak

for {
    select {
    case t := <-ticker.C:
        fmt.Println("tick at", t)
    case <-done:
        return
    }
}
```

> ***Interview tip:*** *`time.After` in a loop creates a new timer on every iteration — memory leak. Use `time.NewTimer` with `Reset` or `time.NewTicker` for repeating work.*

Q27. What is a channel as a mutex pattern?

Pattern: Channel-based lock

Difficulty: Hard

A buffered channel of capacity 1 can act as a binary semaphore (mutex). Send to lock, receive to unlock.

```
type Mutex chan struct{}

func NewMutex() Mutex { return make(Mutex, 1) }
func (m Mutex) Lock()   { m <- struct{}{} }
func (m Mutex) Unlock() { <-m }

m := NewMutex()
m.Lock()
// critical section
m.Unlock()
```

> ***Interview tip:*** *This is mostly educational. `sync.Mutex` is faster and clearer. But it demonstrates that channels are a general synchronisation primitive.*

Q28. Implement an OR-done channel — stop receiving when any of N channels closes.

Pattern: OR-done

Difficulty: Hard

Recursively select between pairs of channels. When any closes, close the output. This is a tree reduction of N channels.

```
func orDone(channels ...<-chan struct{}) <-chan struct{} {
    if len(channels) == 0 { return nil }
    if len(channels) == 1 { return channels[0] }

    out := make(chan struct{})
    go func() {
        defer close(out)
        switch len(channels) {
        case 2:
            select {
            case <-channels[0]:
            case <-channels[1]:
            }
        default:
            select {
            case <-channels[0]:
            case <-channels[1]:
            case <-channels[2]:
            case <-orDone(append(channels[3:], out)...):
            }
        }
    }()
    return out
}
```

> ***Interview tip:*** *This pattern appears in the Go concurrency book by Katherine Cox-Buday. Knowing it shows deep Go expertise.*

Runtime & Memory

Q29. How does Go garbage collection work?

Pattern: GC internals | **Difficulty: Medium**

Go uses a concurrent tri-color mark-and-sweep GC. Most GC work runs concurrently with your program. STW (stop-the-world) pauses are typically under 1ms.

```
// Trigger GC manually (testing only)
runtime.GC()

// Inspect GC stats
var stats runtime.MemStats
runtime.ReadMemStats(&stats)
fmt.Printf("GC cycles: %d\n", stats.NumGC)
fmt.Printf("Heap: %d KB\n", stats.HeapAlloc/1024)
fmt.Printf("NextGC: %d KB\n", stats.NextGC/1024)
```

Interview tip: *"GOGC=100 means GC triggers when heap size doubles from previous GC. GOGC=off disables GC (for benchmarks)."*

Q30. What is escape analysis in Go?

Pattern: Memory allocation | **Difficulty: Medium**

The compiler decides at compile time whether a variable lives on the stack or heap. Variables that outlive their function "escape" to the heap and are garbage collected.

```
// Stack allocated — does not escape
func stackAlloc() int {
    x := 42
    return x // value copied, x stays on stack
}

// Heap allocated — pointer escapes
func heapAlloc() *int {
    x := 42
    return &x // x escapes to heap
}
```

```
# Check escape analysis
go build -gcflags="-m" ./...
# Output: ./main.go:8:2: moved to heap: x
```

Interview tip: *Interface boxing always causes heap allocation. Avoid in hot paths.*

Q31. What is the difference between stack and heap allocation?

Pattern: Memory model | **Difficulty: Easy**

Stack allocation: O(1) — just a pointer bump, no GC involvement. Heap allocation: slower, adds GC pressure. Go prefers stack allocation when possible.

```
// Force heap allocation: return pointer
func newInt(v int) *int { return &v }

// Stack allocation: value semantics
func sumSlice(s []int) int {
    total := 0  // stack
    for _, v := range s { total += v }
    return total
```

```
}
```

> ***Interview tip:*** *Large structs passed by value get copied on stack. Pass pointers for structs > ~100 bytes to avoid copy cost — but consider escape analysis.*

Q32. What is GOGC and how does it affect performance?

Pattern: GC tuning | **Difficulty: Medium**

`GOGC` controls the GC trigger threshold — the percentage by which the heap can grow before the next GC. Default is 100 (heap doubles). Lower = more frequent GC = more CPU. Higher = less frequent GC = more memory.

```
// In code
debug.SetGCPercent(200) // GC when heap triples (less frequent)
debug.SetGCPercent(50)  // GC at 50% growth (more frequent)
debug.SetGCPercent(-1)  // Disable GC entirely
```

```
GOGC=200 go run main.go  # environment variable
```

> ***Interview tip:*** *For latency-sensitive services: increase GOGC to reduce GC pauses at the cost of higher memory usage.*

Q33. What is pprof and how do you use it?

Pattern: Profiling | **Difficulty: Medium**

pprof is Go's built-in profiler. It profiles CPU, memory (heap), goroutines, and blocking.

```
import _ "net/http/pprof"  // registers /debug/pprof/ endpoints

// In your server
go http.ListenAndServe(":6060", nil)

// For tests
go test -bench=. -cpuprofile cpu.prof -memprofile mem.prof ./...
go tool pprof cpu.prof
// (pprof) top10
// (pprof) web        → opens flamegraph
// (pprof) list main  → annotated source
```

> ***Interview tip:*** *"I profile before optimising. pprof's flamegraph shows where CPU time is actually spent — without it, optimisation is guesswork."*

Q34. How do you reduce memory allocations in a hot path?

Pattern: Allocation optimisation | **Difficulty: Hard**

Use `sync.Pool` for temporary objects, avoid interface boxing, use value types, pre-allocate slices with `make([]T, 0, n)`, reuse buffers.

```
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

func processRequest(data []byte) string {
    buf := bufPool.Get().(*bytes.Buffer)
    defer func() {
        buf.Reset()
        bufPool.Put(buf)
    }()
    buf.Write(data)
```

```
    // ... process
    return buf.String()
}

// Pre-allocate
result := make([]int, 0, len(input)) // avoids reallocations
```

> ***Interview tip:*** *Use `go test -bench=. -benchmem` to see allocations per operation. Then target the biggest contributors.*

Q35. What is the unsafe package and when would you use it?

Pattern: Low-level access

Difficulty: Hard

`unsafe` bypasses Go's type safety — direct memory access, pointer arithmetic, size/alignment queries. Used in performance-critical serialisation, CGo interop, and system programming.

```
import "unsafe"

type MyStruct struct {
    A int32
    B int64
    C bool
}

fmt.Println(unsafe.Sizeof(MyStruct{}))     // 24 (with padding)
fmt.Println(unsafe.Alignof(MyStruct{}))    // 8
fmt.Println(unsafe.Offsetof(MyStruct{}.B)) // 8

// Zero-copy string to []byte (read-only!)
func stringToBytes(s string) []byte {
    return unsafe.Slice(unsafe.StringData(s), len(s))
}
```

> ***Interview tip:*** *"unsafe is appropriate when: (1) you've profiled and proven it necessary, (2) the code is isolated, (3) tests cover the behaviour extensively."*

Q36. How does slice internals work in Go?

Pattern: Slice internals

Difficulty: Medium

A slice header is 3 words: pointer to underlying array, length, capacity. Slicing creates a new header pointing into the same array — no copy.

```
a := []int{1, 2, 3, 4, 5}
b := a[1:4]          // len=3, cap=4, points into a's array
b[0] = 99            // modifies a[1] as well!
fmt.Println(a)       // [1, 99, 3, 4, 5]

// Copy to get independent slice
c := make([]int, len(b))
copy(c, b)
c[0] = 0             // does NOT affect a
```

> ***Interview tip:*** *"Slices share underlying arrays. This is efficient but can cause surprising aliasing bugs. When in doubt, copy."*

Q37. What is the difference between make and new in Go?

Pattern: Allocation primitives

Difficulty: Easy

`new(T)` allocates zeroed memory for type T, returns `*T`. `make(T, ...)` creates and initialises slices, maps, channels — returns T (not a pointer). Cannot use `make` for arbitrary types.

```
p := new(int)       // *int, *p == 0
s := make([]int, 5) // []int{0,0,0,0,0}
m := make(map[string]int) // empty initialised map
ch := make(chan int, 10)  // buffered channel

// new is rarely needed in idiomatic Go
// prefer: var x T or x := T{}
```

Interview tip: *"You almost never need `new`. Prefer `var x T` for zero values or `&T;{}` for pointers to structs."*

Q38. How does map internals work in Go?

Pattern: Map internals

Difficulty: Medium

Go maps are hash tables with open addressing. Buckets hold 8 key-value pairs. On high load factor, buckets are incremented. Maps are not safe for concurrent use — use `sync.Map` or a mutex.

```
// NOT safe for concurrent access
m := map[string]int{"a": 1}

// Safe option 1: sync.Map (for high concurrency)
var sm sync.Map
sm.Store("key", 42)
v, ok := sm.Load("key")

// Safe option 2: RWMutex
type SafeMap struct {
    mu sync.RWMutex
    m  map[string]int
}

// Iteration order is random — by design
for k, v := range m { fmt.Println(k, v) }
```

Interview tip: *"Map reads are safe to do concurrently in Go — but only if no goroutine is writing. Any concurrent write requires a lock."*

Q39. What is a string in Go and how does it differ from []byte?

Pattern: String internals

Difficulty: Easy

A string is an immutable sequence of bytes (not characters). It's a 2-word header: pointer + length. `[]byte` is mutable, 3-word header: pointer + length + capacity. Conversion copies the data.

```
s := "Hello, ■■"
fmt.Println(len(s))         // 13 bytes (not 9 characters)
fmt.Println([]rune(s))      // [72 101 108 108 111 44 32 19990 30028]

// Iterate runes (Unicode code points)
for i, r := range s {
    fmt.Printf("%d: %c (%d)\n", i, r, r)
}
```

```
// []byte ↔ string conversion copies
b := []byte(s)   // copy
s2 := string(b)  // copy
```

> ***Interview tip:*** *"Use `strings.Builder` for efficient string concatenation — it avoids O(n²) copying from repeated `+` concatenation."*

Q40. What is init() in Go and when is it called?

Pattern: Initialisation | **Difficulty: Medium**

`init()` is called automatically after all package-level variables are initialised, before `main()`. A package can have multiple `init()` functions. They run in the order they appear, in dependency order.

```
var config = loadConfig() // package-level var, initialised first

func init() {
    // runs after package-level vars, before main()
    log.SetFlags(log.LstdFlags | log.Lshortfile)
    log.Println("package initialised")
}

func init() { // multiple init() allowed
    setupMetrics()
}

func main() {
    // init() already ran
}
```

> ***Interview tip:*** *Avoid complex logic in `init()`. It runs at import time, can't be tested easily, and creates hidden dependencies. Prefer explicit initialisation functions.*

Interfaces & Types

Q41. How do interfaces work in Go? What is implicit implementation?

Pattern: Interface fundamentals | **Difficulty: Easy**

A type satisfies an interface by having all required methods — no `implements` keyword. Interfaces are two-word pairs internally: type info (itab) + data pointer.

```
type Writer interface {
    Write(p []byte) (n int, err error)
}

// Any type with Write() satisfies Writer implicitly
type MyWriter struct{}
func (m MyWriter) Write(p []byte) (int, error) {
    fmt.Println(string(p))
    return len(p), nil
}

var w Writer = MyWriter{} // no declaration needed
```

> ***Interview tip:*** *"Accept interfaces, return concrete types." — the most important Go design principle.*

Q42. What is the empty interface (any) and its pitfalls?

Pattern: Type system

Difficulty: Medium

`interface{}` (aliased as `any` in Go 1.18+) holds any value. Pitfalls: type information is lost, boxing causes heap allocation, type safety is bypassed.

```
func print(v any) {
    switch t := v.(type) {
    case int:    fmt.Println("int:", t)
    case string: fmt.Println("string:", t)
    default:     fmt.Printf("unknown: %T\n", t)
    }
}

// Type assertion
var v any = "hello"
s, ok := v.(string) // safe assertion — ok=false on failure
s = v.(string)      // panics if v is not string
```

> ***Interview tip:*** *"Avoid `any` in new code — use generics instead. But know it for JSON unmarshalling and reflection questions."*

Q43. What is the nil interface vs nil pointer inside interface bug?

Pattern: Interface nil trap

Difficulty: Hard

A nil interface has both type and value nil. A nil pointer inside a non-nil interface has type info set but value nil. Comparing to `nil` returns `false` — the classic Go gotcha.

```
type MyError struct{ msg string }
func (e *MyError) Error() string { return e.msg }

func getError(fail bool) error {
    var err *MyError = nil
    if fail { err = &MyError{"oops"} }
    return err // BUG: non-nil interface, nil value!
}

err := getError(false)
fmt.Println(err == nil) // false! interface has type *MyError

// Fix: return typed nil directly
func getErrorFixed(fail bool) error {
    if fail { return &MyError{"oops"} }
    return nil // truly nil interface
}
```

> ***Interview tip:*** *This is the most common Go interview trap. Always return `nil` as the interface type, never as a typed nil pointer.*

Q44. What is a type assertion vs type switch?

Pattern: Type inspection

Difficulty: Easy

Type assertion extracts a specific type from an interface. Type switch handles multiple types cleanly.

```
var i interface{} = "hello"
```

```
// Type assertion (panics on failure without ok)
s, ok := i.(string)
if ok { fmt.Println("string:", s) }

// Type switch
switch v := i.(type) {
case int:
    fmt.Printf("int: %d\n", v)
case string:
    fmt.Printf("string: %s\n", v)
case []byte:
    fmt.Printf("bytes: %s\n", v)
default:
    fmt.Printf("unknown: %T\n", v)
}
```

> ***Interview tip:*** *"Always use the two-return form `v, ok := x.(T)` unless you are certain of the type. Panics in production are bad."*

Q45. What is method set in Go?

Pattern: Method receivers

Difficulty: Medium

The method set of a type determines which interfaces it satisfies. Value receiver methods belong to both `T` and `*T`. Pointer receiver methods belong only to `*T`.

```
type Animal struct{ name string }

func (a Animal) Speak() string  { return a.name + " speaks" }  // value receiver
func (a *Animal) Rename(n string) { a.name = n }               // pointer receiver

// Dog satisfies Speaker (value receiver — both T and *T have it)
type Speaker interface { Speak() string }

var s Speaker = Animal{"Cat"}    // OK
var s2 Speaker = &Animal{"Dog"}  // OK

// Renamer interface needs pointer receiver — only *Animal satisfies it
type Renamer interface { Rename(string) }
// var r Renamer = Animal{"Cat"} // COMPILE ERROR
var r Renamer = &Animal{"Cat"}   // OK
```

> ***Interview tip:*** *"If in doubt, use pointer receivers. They're consistent, allow mutation, and avoid copies of large structs."*

Q46. What is embedding in Go?

Pattern: Composition over inheritance

Difficulty: Medium

Embedding promotes methods and fields of the embedded type. It is composition, not inheritance — there is no `super` call, no override, no polymorphism.

```
type Animal struct { name string }
func (a Animal) Speak() string { return a.name }

type Dog struct {
    Animal          // embedded — Dog gets Speak()
```

```
    Breed string
}

func (d Dog) Fetch() string { return d.name + " fetches!" }

d := Dog{Animal{"Rex"}, "Lab"}
fmt.Println(d.Speak())  // promoted from Animal
fmt.Println(d.Fetch())  // Dog's own method
fmt.Println(d.name)     // promoted field
```

> ***Interview tip:*** *"Embedding is Go's answer to inheritance. But there's no polymorphism — `d.Speak()` always calls Animal's Speak, even if Dog defines one (Dog's would shadow it)."*

Q47. How do you implement the Stringer interface?

Pattern: fmt.Stringer

Difficulty: Easy

Implement `String() string` to control how a type is printed by `fmt`.

```
type Point struct{ X, Y int }

func (p Point) String() string {
    return fmt.Sprintf("(%d, %d)", p.X, p.Y)
}

p := Point{3, 4}
fmt.Println(p)         // (3, 4)
fmt.Printf("%v\n", p)  // (3, 4)
fmt.Printf("%s\n", p)  // (3, 4)
```

> ***Interview tip:*** *Also know `error` (single-method interface), `io.Reader`, `io.Writer`, `sort.Interface` — these are the most important interfaces in the stdlib.*

Q48. What is the difference between pointer and value receivers?

Pattern: Receiver types

Difficulty: Easy

Value receiver: gets a copy, cannot modify original. Pointer receiver: gets a reference, can modify original. Use value for small, immutable data; pointer for mutating state or large structs.

```
type Counter struct{ count int }

// Value receiver — copy, cannot mutate
func (c Counter) Value() int { return c.count }

// Pointer receiver — can mutate
func (c *Counter) Increment() { c.count++ }

c := Counter{}
c.Increment()           // Go auto-takes &c
fmt.Println(c.Value())  // 1
```

> ***Interview tip:*** *"Be consistent — if any method uses pointer receiver, use pointer receiver for all methods on that type."*

Q49. How does Go handle interface satisfaction at compile time?

Pattern: Compile-time interface check

Difficulty: Medium

Use a blank identifier assignment to assert interface satisfaction at compile time. This produces a clear error message rather than a runtime panic.

```
type MyHandler struct{}
func (h *MyHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {}

// Compile-time check: *MyHandler implements http.Handler
var _ http.Handler = (*MyHandler)(nil)
// If MyHandler doesn't implement the interface → compile error
```

> ***Interview tip:*** *This is a common pattern in Go libraries. `var _ InterfaceName = (*ConcreteType)(nil)` is idiomatic for type assertions at compile time.*

Q50. What is duck typing in Go?

Pattern: Structural typing

Difficulty: Easy

Go uses structural typing — if a type has the right methods, it satisfies the interface, regardless of declaration. This is sometimes called "duck typing" though it's checked at compile time.

```
// Nothing declares it implements Quacker
type Duck struct{}
func (d Duck) Quack() { fmt.Println("Quack!") }

type Robot struct{}
func (r Robot) Quack() { fmt.Println("*mechanical quack*") }

type Quacker interface{ Quack() }

func makeItQuack(q Quacker) { q.Quack() }

makeItQuack(Duck{})   // works
makeItQuack(Robot{})  // works — structural typing
```

> ***Interview tip:*** *"Go's interfaces enable decoupled design. You can define an interface against code you don't own, without modifying it."*

Q51. How do you mock interfaces in Go for testing?

Pattern: Testability

Difficulty: Medium

Define thin interfaces over dependencies. In tests, provide a mock implementation. No reflection or magic needed.

```
type UserStore interface {
    GetUser(id int) (*User, error)
    SaveUser(u *User) error
}

// Mock for testing
type MockUserStore struct {
    users map[int]*User
    err   error
}

func (m *MockUserStore) GetUser(id int) (*User, error) {
```

```
    return m.users[id], m.err
}

func (m *MockUserStore) SaveUser(u *User) error { return m.err }

// Test
func TestService(t *testing.T) {
    mock := &MockUserStore{
        users: map[int]*User{1: {Name: "Alice"}},
    }
    svc := NewUserService(mock) // inject mock
    user, err := svc.GetUser(1)
    if err != nil || user.Name != "Alice" {
        t.Fatal("unexpected result")
    }
}
```

Interview tip: *"Interfaces for testability — keep interfaces small (1–3 methods). Wide interfaces are hard to mock."*

Q52. What is the io.Reader and io.Writer interface pattern?

Pattern: io interfaces | **Difficulty: Medium**

`io.Reader` and `io.Writer` are the most important interfaces in Go. They compose — `io.TeeReader`, `io.MultiWriter`, `io.Pipe`, `bufio.Scanner` all work with any Reader/Writer.

```
// io.Reader — read from anything
func countBytes(r io.Reader) (int64, error) {
    return io.Copy(io.Discard, r)
}

// io.Writer — write to anything
func writeJSON(w io.Writer, v any) error {
    return json.NewEncoder(w).Encode(v)
}

// Works with files, network, bytes, gzip, etc.
countBytes(os.Stdin)
countBytes(bytes.NewReader(data))
countBytes(resp.Body) // http.Response

writeJSON(os.Stdout, myStruct)
writeJSON(buf, myStruct)        // bytes.Buffer
writeJSON(gzipWriter, myStruct) // compressed
```

Interview tip: *"io.Reader and io.Writer are why Go code is so composable. Any function accepting these works with all data sources and sinks."*

Error Handling

Q53. What is idiomatic error handling in Go?

Pattern: Error values | **Difficulty: Easy**

Go uses explicit error return values. Callers check `err != nil`. This forces you to think about every failure path.

```
func divide(a, b float64) (float64, error) {
```

```
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
    log.Printf("error: %v", err)
    return
}
fmt.Println(result)
```

> ***Interview tip:*** *"In Go, errors are values — treat them like any other return value, not as exceptional control flow."*

Q54. What is error wrapping with %w?

Pattern: Error wrapping

Difficulty: Medium

`fmt.Errorf` with `%w` wraps an error, preserving the chain. `errors.Is` checks if any error in the chain matches. `errors.As` checks if any error in the chain can be assigned to a target type.

```
var ErrNotFound = errors.New("not found")

func findUser(id int) error {
    return fmt.Errorf("findUser %d: %w", id, ErrNotFound)
}

err := findUser(42)
fmt.Println(errors.Is(err, ErrNotFound)) // true

// errors.As for typed errors
type ValidationErr struct{ Field string }
func (e *ValidationErr) Error() string { return "invalid: " + e.Field }

var ve *ValidationErr
if errors.As(err, &ve) {
    fmt.Println("field:", ve.Field)
}
```

> ***Interview tip:*** *"Always wrap errors with context: `fmt.Errorf('operation: %w', err)`. Never return bare errors without context."*

Q55. When should you use panic vs error?

Pattern: Error vs panic

Difficulty: Hard

Use `error` for expected failure conditions callers should handle. Use `panic` only for unrecoverable programming bugs or failed invariants. Libraries should never panic for user-facing errors.

```
// OK to panic: programmer error
func mustPositive(n int) int {
    if n <= 0 { panic(fmt.Sprintf("expected positive, got %d", n)) }
    return n
}

// Recover at goroutine boundary
func safeGo(f func()) {
```

```
    go func() {
        defer func() {
            if r := recover(); r != nil {
                log.Printf("recovered panic: %v", r)
            }
        }()
        f()
    }()
}
```

> ***Interview tip:*** *"Panic is for impossible states. Error is for expected failures. If a user can trigger it, return an error."*

Q56. How do you create custom error types in Go?

Pattern: Custom errors

Difficulty: Medium

Implement the `error` interface (`Error() string`). Add fields for structured error information. Use `errors.As` to extract them.

```
type HTTPError struct {
    StatusCode int
    Message    string
}

func (e *HTTPError) Error() string {
    return fmt.Sprintf("HTTP %d: %s", e.StatusCode, e.Message)
}

func fetchData(url string) error {
    resp, err := http.Get(url)
    if err != nil { return fmt.Errorf("fetch: %w", err) }
    if resp.StatusCode != 200 {
        return &HTTPError{
            StatusCode: resp.StatusCode,
            Message:    "unexpected status",
        }
    }
    return nil
}

err := fetchData("https://example.com")
var httpErr *HTTPError
if errors.As(err, &httpErr) {
    fmt.Println("status code:", httpErr.StatusCode)
}
```

> ***Interview tip:*** *"Prefer wrapping standard errors with context over creating custom types for simple cases. Create custom types when callers need to inspect error fields."*

Q57. What is the errors.New vs fmt.Errorf difference?

Pattern: Error creation

Difficulty: Easy

`errors.New` creates a simple error with a static message. `fmt.Errorf` creates a formatted error — use `%w` to wrap another error.

```
// errors.New — static sentinel errors
var ErrTimeout = errors.New("operation timed out")
```

```
var ErrNotFound = errors.New("not found")

// fmt.Errorf — dynamic errors with context
func getUser(id int) error {
    return fmt.Errorf("getUser(%d): %w", id, ErrNotFound)
}

// Sentinel comparison
errors.Is(err, ErrNotFound) // true if ErrNotFound in chain
```

> ***Interview tip:*** *"Sentinel errors (`var ErrX = errors.New(...)`) should be package-level vars so callers can compare with `errors.Is`."*

Q58. How do you handle errors in goroutines?

Pattern: Goroutine error propagation

Difficulty: Hard

Goroutines can't return errors. Use channels to propagate errors back to the caller.

```
func doWork(ctx context.Context) <-chan error {
    errCh := make(chan error, 1)
    go func() {
        defer close(errCh)
        if err := riskyOperation(ctx); err != nil {
            errCh <- err
        }
    }()
    return errCh
}

// Collect errors from multiple goroutines
errCh := make(chan error, len(tasks))
for _, t := range tasks {
    go func(task Task) {
        errCh <- task.Run()
    }(t)
}

for range tasks {
    if err := <-errCh; err != nil {
        log.Println("task error:", err)
    }
}
```

> ***Interview tip:*** *"Also know `errgroup.Group` from `golang.org/x/sync/errgroup` — it handles exactly this pattern with cancellation."*

Q59. What is errgroup and how do you use it?

Pattern: Error groups

Difficulty: Medium

`errgroup.Group` runs goroutines and collects the first non-nil error. Optionally cancels all goroutines when one fails.

```
import "golang.org/x/sync/errgroup"

func fetchAll(urls []string) error {
    g, ctx := errgroup.WithContext(context.Background())
```

```
    results := make([]string, len(urls))
    for i, url := range urls {
        i, url := i, url // capture loop vars
        g.Go(func() error {
            r, err := fetchWithCtx(ctx, url)
            if err != nil { return err }
            results[i] = r
            return nil
        })
    }

    if err := g.Wait(); err != nil {
        return fmt.Errorf("fetchAll: %w", err)
    }
    return nil
}
```

> ***Interview tip:*** *"errgroup is cleaner than manual WaitGroup + channel error collection. Use it in any concurrent fan-out with error propagation."*

Q60. How do you implement retry with exponential backoff in Go?

Pattern: Resilience | **Difficulty: Medium**

Retry transient errors with increasing delays. Cap the delay and add jitter to prevent thundering herd.

```
func retryWithBackoff(ctx context.Context, op func() error) error {
    backoff := 100 * time.Millisecond
    maxBackoff := 30 * time.Second
    maxRetries := 5

    for i := 0; i < maxRetries; i++ {
        err := op()
        if err == nil { return nil }

        // Add jitter: backoff ± 10%
        jitter := time.Duration(rand.Int63n(int64(backoff / 5)))
        wait := backoff + jitter
        if wait > maxBackoff { wait = maxBackoff }

        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(wait):
        }

        backoff *= 2
    }
    return fmt.Errorf("operation failed after %d retries", maxRetries)
}
```

> ***Interview tip:*** *"Always respect context cancellation in retry loops. And add jitter — without it, all clients retry at the same time, causing a thundering herd."*

Context

Q61. What is context.Context and why is it the first argument?

Pattern: Context fundamentals

Difficulty: Easy

`context.Context` carries deadlines, cancellation signals, and request-scoped values across API boundaries. Passed as the first argument so any long-running operation can be cancelled.

```
func fetchUser(ctx context.Context, id int) (*User, error) {
    req, _ := http.NewRequestWithContext(ctx, "GET",
        fmt.Sprintf("/users/%d", id), nil)
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return nil, err // includes context.Canceled
    }
    defer resp.Body.Close()
    // ...
}

ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
user, err := fetchUser(ctx, 42)
```

> ***Interview tip:*** *"Always `defer cancel()` immediately after creating a cancellable context. Forgetting causes goroutine leaks."*

Q62. What is the difference between WithCancel, WithTimeout, and WithDeadline?

Pattern: Context variants

Difficulty: Medium

`WithCancel`: you control when cancelled. `WithTimeout`: cancelled after a duration. `WithDeadline`: cancelled at an absolute time. Cancelling a parent cancels all children.

```
// WithCancel — manual cancel
ctx, cancel := context.WithCancel(parent)
defer cancel()

// WithTimeout — duration from now
ctx, cancel := context.WithTimeout(parent, 3*time.Second)
defer cancel()

// WithDeadline — absolute time
ctx, cancel := context.WithDeadline(parent, time.Now().Add(3*time.Second))
defer cancel()

// Check why context ended
<-ctx.Done()
fmt.Println(ctx.Err()) // context.DeadlineExceeded or Canceled
```

> ***Interview tip:*** *"WithTimeout is the most common in production. Set it at the service boundary and propagate through layers."*

Q63. How do you store and retrieve values in context?

Pattern: Context values

Difficulty: Medium

Use typed keys (not string literals) to avoid collisions between packages. Context values are for request-scoped metadata, not function parameters.

```
type contextKey string
const requestIDKey contextKey = "requestID"

func WithRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}

func GetRequestID(ctx context.Context) (string, bool) {
    id, ok := ctx.Value(requestIDKey).(string)
    return id, ok
}

// In HTTP middleware
func middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ctx := WithRequestID(r.Context(), uuid.New().String())
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

Interview tip: *"Context values are for metadata — trace IDs, auth tokens. If removing a value breaks business logic, it should be a function parameter."*

Q64. How do you propagate context in HTTP clients?

Pattern: HTTP context

Difficulty: Medium

Use `http.NewRequestWithContext` to attach context to outgoing requests. When context is cancelled, the request is cancelled too.

```
func callExternalAPI(ctx context.Context, payload any) (*Response, error) {
    body, _ := json.Marshal(payload)

    req, err := http.NewRequestWithContext(ctx, "POST",
        "https://api.example.com/endpoint",
        bytes.NewReader(body))
    if err != nil { return nil, err }

    req.Header.Set("Content-Type", "application/json")

    client := &http.Client{Timeout: 10 * time.Second}
    resp, err := client.Do(req)
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            return nil, fmt.Errorf("API call timed out: %w", err)
        }
        return nil, fmt.Errorf("API call failed: %w", err)
    }
    defer resp.Body.Close()
    // parse response...
}
```

Interview tip: *"Set a timeout both on the context AND on `http.Client`. The client timeout is a safety net; context timeout propagates cancellation semantics."*

Q65. What is context.Background() vs context.TODO()?

Pattern: Root contexts

Difficulty: Easy

Both are empty, non-cancellable contexts. `Background()` is the root context for all requests. `TODO()` is a placeholder when you're unsure which context to use (signals work in progress).

```
// Background — use at program entry points
ctx := context.Background()

// In main:
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

// TODO — marks code that needs context plumbing later
func oldFunction() {
    ctx := context.TODO() // come back and wire up real context
    doWork(ctx)
}
```

> ***Interview tip:*** *"`context.TODO()` is a code smell that signals: 'this should receive a context from the caller but doesn't yet.' Grep for it to find tech debt."*

Q66. How do you detect context cancellation inside a long loop?

Pattern: Cancellation detection

Difficulty: Medium

Check `ctx.Done()` in a `select` at each iteration, or periodically via `ctx.Err()` for CPU-intensive work.

```
func processItems(ctx context.Context, items []Item) error {
    for _, item := range items {
        // Check cancellation each iteration
        select {
        case <-ctx.Done():
            return fmt.Errorf("processing cancelled: %w", ctx.Err())
        default:
        }

        if err := process(item); err != nil {
            return err
        }
    }
    return nil
}

// For tight CPU loops — check every N iterations
for i, item := range items {
    if i%100 == 0 {
        if err := ctx.Err(); err != nil { return err }
    }
    heavyComputation(item)
}
```

> ***Interview tip:*** *"The `select` with `default` is non-blocking — it checks if Done without waiting. This is the standard pattern."*

Q67. What are the rules for context in Go?

Pattern: Context best practices

Difficulty: Medium

Five rules: (1) Pass context as first parameter. (2) Never store context in a struct. (3) Never pass nil context. (4) Only use context values for request-scoped data. (5) Always cancel when done.

```
// WRONG — storing context in a struct
type Handler struct {
    ctx context.Context // BAD
}

// CORRECT — pass context to each method
type Handler struct{}
func (h *Handler) Handle(ctx context.Context, req Request) error {
    return process(ctx, req)
}

// WRONG — passing nil
doWork(nil)

// CORRECT — use TODO if no context available
doWork(context.TODO())
```

> ***Interview tip:*** *"These rules are from the official Go blog. Mentioning them by number shows you know the documentation."*

Generics

Q68. What are generics in Go and when were they introduced?

Pattern: Generics basics

Difficulty: Easy

Generics (type parameters) introduced in Go 1.18 (March 2022). Write functions and types that work with any type satisfying a constraint, without losing type safety.

```
// Generic function
func Map[T, U any](slice []T, f func(T) U) []U {
    result := make([]U, len(slice))
    for i, v := range slice {
        result[i] = f(v)
    }
    return result
}

nums := []int{1, 2, 3}
strs := Map(nums, strconv.Itoa) // ["1","2","3"]

// Generic type
type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 { var zero T; return zero, false }
```

```
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}
```

> ***Interview tip:*** *"Use generics when you find yourself copy-pasting the same logic for different types. Not for everything."*

Q69. What is a type constraint in Go generics?

Pattern: Constraints | **Difficulty: Medium**

Constraints restrict which types can be type arguments. `any` allows all types. `comparable` allows ==. Custom constraints use interface syntax with type unions.

```
// Built-in constraint
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice {
        if v == item { return true }
    }
    return false
}

// Custom numeric constraint
type Number interface {
    ~int | ~int32 | ~int64 | ~float32 | ~float64
}

func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums { total += n }
    return total
}

Sum([]int{1, 2, 3})        // 6
Sum([]float64{1.1, 2.2})   // 3.3
```

> ***Interview tip:*** *"`~int` means any type whose underlying type is int — includes type aliases like `type MyInt int`. This is the key to flexible constraints."*

Q70. What is the ordered constraint?

Pattern: Ordered types | **Difficulty: Easy**

`constraints.Ordered` (from `golang.org/x/exp/constraints`) covers all types that support <, >, <=, >=. Includes integers, floats, and strings.

```
import "golang.org/x/exp/constraints"

func Min[T constraints.Ordered](a, b T) T {
    if a < b { return a }
    return b
}

func Max[T constraints.Ordered](a, b T) T {
    if a > b { return a }
    return b
}
fmt.Println(Min(3, 5))     // 3
```

```
fmt.Println(Min("a", "b")) // "a"
```

> ***Interview tip:*** *"In Go 1.21+, `min` and `max` are built-in builtins. Before that, you'd write a generic version or use the x/exp package."*

Q71. How do generics affect performance in Go?

Pattern: Generic internals

Difficulty: Hard

Go generics use GCShape stenciling — similar concrete types share the same code. Pointer types and interface types each get one instantiation. Value types may get specialised code. Generally comparable to non-generic code.

```
// Both use the same generated code (GCShape = pointer)
s1 := Stack[*int]{}
s2 := Stack[*string]{}

// These may get separate instantiations (different GCShapes)
s3 := Stack[int]{}
s4 := Stack[float64]{}

// Benchmark to check — generics overhead is usually negligible
func BenchmarkGenericMin(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Min(rand.Int(), rand.Int())
    }
}
```

> ***Interview tip:*** *"Go generics are not C++ templates — no code bloat. GCShape stenciling keeps binary size controlled."*

Q72. Implement a generic filter and reduce function.

Pattern: Functional generics

Difficulty: Medium

```
func Filter[T any](slice []T, pred func(T) bool) []T {
    var result []T
    for _, v := range slice {
        if pred(v) { result = append(result, v) }
    }
    return result
}

func Reduce[T, U any](slice []T, init U, fn func(U, T) U) U {
    result := init
    for _, v := range slice {
        result = fn(result, v)
    }
    return result
}

// Usage
nums := []int{1, 2, 3, 4, 5, 6}
evens := Filter(nums, func(n int) bool { return n%2 == 0 }) // [2,4,6]
sum := Reduce(nums, 0, func(acc, n int) int { return acc + n }) // 21
```

> ***Interview tip:*** *"These patterns come from functional programming. In Go, they're useful but verbose compared to Python/Haskell. Generics make them type-safe."*

Q73. What are interface constraints vs type constraints?

Pattern: Advanced constraints

Difficulty: Hard

Type constraints restrict to specific types (`~int | ~string`). Interface constraints require specific methods. You can combine both in a single constraint.

```
// Method constraint
type Stringer interface {
    String() string
}

// Type + method constraint
type StringerOrNumber interface {
    ~int | ~float64 | Stringer
}

// Combined: must be ordered AND have a String method
type OrderedStringer interface {
    constraints.Ordered
    String() string
}

func PrintSorted[T OrderedStringer](items []T) {
    sort.Slice(items, func(i, j int) bool {
        return items[i] < items[j]
    })
    for _, v := range items { fmt.Println(v.String()) }
}
```

Interview tip: *"Union types in constraints only work for basic operations (`+`, `<`, etc.), not method calls. To call methods, use the method constraint form."*

Q74. How do you write a generic set data structure?

Pattern: Generic data structures

Difficulty: Medium

```
type Set[T comparable] struct {
    items map[T]struct{}
}

func NewSet[T comparable](vals ...T) *Set[T] {
    s := &Set[T]{items: make(map[T]struct{})}
    for _, v := range vals { s.Add(v) }
    return s
}

func (s *Set[T]) Add(v T)            { s.items[v] = struct{}{} }
func (s *Set[T]) Remove(v T)         { delete(s.items, v) }
func (s *Set[T]) Contains(v T) bool  { _, ok := s.items[v]; return ok }
func (s *Set[T]) Len() int           { return len(s.items) }

func (s *Set[T]) Intersection(other *Set[T]) *Set[T] {
    result := NewSet[T]()
    for v := range s.items {
        if other.Contains(v) { result.Add(v) }
```

```
    }
    return result
}

// Usage
s1 := NewSet(1, 2, 3, 4)
s2 := NewSet(3, 4, 5, 6)
inter := s1.Intersection(s2) // {3, 4}
```

> ***Interview tip:*** *"Generic Set is a great LLD warm-up. Requires `comparable` constraint because map keys must be comparable."*

Testing & Benchmarking

Q75. How do you write a table-driven test in Go?

Pattern: Table-driven tests | **Difficulty: Easy**

Define a slice of test cases with input and expected output. Loop calling `t.Run` for sub-tests.

```
func TestDivide(t *testing.T) {
    tests := []struct {
        name    string
        a, b    float64
        want    float64
        wantErr bool
    }{
        {"normal", 10, 2, 5, false},
        {"divide by zero", 10, 0, 0, true},
        {"negative", -6, 2, -3, false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := divide(tt.a, tt.b)
            if (err != nil) != tt.wantErr {
                t.Errorf("err=%v, wantErr=%v", err, tt.wantErr)
            }
            if !tt.wantErr && got != tt.want {
                t.Errorf("got %v, want %v", got, tt.want)
            }
        })
    }
}
```

> ***Interview tip:*** *"Run a single sub-test: `go test -run TestDivide/divide_by_zero`. Essential for debugging."*

Q76. How do you write benchmarks in Go?

Pattern: Benchmarking | **Difficulty: Medium**

Benchmarks use `testing.B`. Run with `go test -bench=. -benchmem`. `b.N` is the number of iterations the framework determines.

```
func BenchmarkDivide(b *testing.B) {
    b.ReportAllocs()
```

```
    for i := 0; i < b.N; i++ {
        divide(float64(i), 2.0)
    }
}

// Run: go test -bench=. -benchmem ./...
// Output: BenchmarkDivide-8   200000000   6.2 ns/op   0 B/op   0 allocs/op
```

> ***Interview tip:*** *"`b.ResetTimer()` after setup code — otherwise setup time is included in benchmark. `b.ReportAllocs()` shows allocations per op."*

Q77. How do you use testify in Go?

Pattern: Test assertions

Difficulty: Easy

`github.com/stretchr/testify` provides `assert` and `require`. `require` stops the test immediately on failure; `assert` continues.

```
import (
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestUser(t *testing.T) {
    user, err := NewUser("Alice", "alice@example.com")
    require.NoError(t, err)          // stop if error
    require.NotNil(t, user)

    assert.Equal(t, "Alice", user.Name)
    assert.Equal(t, "alice@example.com", user.Email)
    assert.True(t, user.IsActive)
}
```

> ***Interview tip:*** *"Use `require` for preconditions (setup), `assert` for actual assertions. This prevents confusing cascade failures."*

Q78. What is test coverage and how do you measure it?

Pattern: Coverage

Difficulty: Easy

```
go test -cover ./...
go test -coverprofile=coverage.out ./...
go tool cover -html=coverage.out  # opens browser
go tool cover -func=coverage.out  # per-function coverage
```

```
// Coverage-friendly code: avoid empty branches
func abs(n int) int {
    if n < 0 { return -n } // test with negative
    return n               // test with positive
}
```

> ***Interview tip:*** *"100% coverage doesn't mean bug-free. Focus on testing behaviour, not lines. Aim for 80%+ on business logic, less on infrastructure glue."*

Q79. How do you test concurrent code in Go?

Pattern: Concurrent testing

Difficulty: Hard

Use `-race` flag for race detection. For deterministic tests, use channels to synchronise assertions. Use `sync.WaitGroup` to wait for goroutines.

```
func TestConcurrentCounter(t *testing.T) {
    c := &Counter{}
    const goroutines = 100
    var wg sync.WaitGroup

    for i := 0; i < goroutines; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c.Increment()
        }()
    }
    wg.Wait()

    if c.Value() != goroutines {
        t.Errorf("got %d, want %d", c.Value(), goroutines)
    }
}
go test -race -count=100 ./... # run 100 times to catch flaky races
```

> ***Interview tip:*** *"Always run concurrent tests with `-race`. And run with `-count=100` — some races are rare and only show up occasionally."*

Q80. How do you use httptest in Go?

Pattern: HTTP testing

Difficulty: Medium

`net/http/httptest` provides `ResponseRecorder` and `Server` for testing HTTP handlers without starting a real server.

```
func TestHandler(t *testing.T) {
    handler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusOK)
        json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
    })

    req := httptest.NewRequest("GET", "/health", nil)
    rr := httptest.NewRecorder()
    handler.ServeHTTP(rr, req)

    assert.Equal(t, http.StatusOK, rr.Code)
    assert.Contains(t, rr.Body.String(), "ok")
}

// Test with a real server
ts := httptest.NewServer(handler)
defer ts.Close()
resp, _ := http.Get(ts.URL + "/health")
```

> ***Interview tip:*** *"httptest.NewServer starts an actual TCP listener. Use it when you need to test HTTP client behaviour (redirects, TLS, etc.)."*

Q81. What is fuzz testing in Go?

Pattern: Fuzz testing

Difficulty: Hard

Fuzz testing (added in Go 1.18) automatically generates inputs to find edge cases and crashes. The fuzzer mutates a seed corpus and tracks code coverage to explore new paths.

```
func FuzzParse(f *testing.F) {
    // Seed corpus — known good inputs
    f.Add("hello")
    f.Add("123")
    f.Add("")

    f.Fuzz(func(t *testing.T, input string) {
        result, err := parse(input)
        if err != nil { return } // errors are acceptable
        // Invariant: re-parsing should give same result
        result2, err2 := parse(result.String())
        if err2 != nil {
            t.Fatalf("re-parse failed: %v", err2)
        }
        if result.String() != result2.String() {
            t.Fatalf("idempotency violated: %q != %q",
                result.String(), result2.String())
        }
    })
}
```

```
go test -fuzz=FuzzParse -fuzztime=30s ./...
```

> ***Interview tip:*** *"Fuzz testing is great for parsers, deserializers, and any code that handles untrusted input. It finds bugs that humans miss."*

Q82. How do you use build tags (build constraints) in Go?

Pattern: Build constraints

Difficulty: Medium

Build constraints control which files are compiled. Used for platform-specific code, testing utilities, and feature flags.

```
//go:build linux
// +build linux   (old syntax, still required for Go < 1.17)

package main

// This file only compiles on Linux
func getProcInfo() string {
    data, _ := os.ReadFile("/proc/self/status")
    return string(data)
}
```

```
//go:build integration

package mypackage_test

// Only runs with: go test -tags=integration ./...
func TestDatabaseIntegration(t *testing.T) {
    db := connectRealDB(t)
    // ...
}
```

> ***Interview tip:*** *"Use `//go:build integration` to separate unit tests from integration tests. CI runs unit tests on every commit, integration tests on merge."*

Go Patterns

Q83. What is the functional options pattern?

Pattern: Functional options

Difficulty: Medium

Pass optional configuration via functions that modify a config struct. Avoids long parameter lists and is backward-compatible when adding new options.

```
type Server struct {
    host    string
    port    int
    timeout time.Duration
}

type Option func(*Server)

func WithPort(p int) Option    { return func(s *Server) { s.port = p } }
func WithTimeout(d time.Duration) Option { return func(s *Server) { s.timeout = d } }

func NewServer(host string, opts ...Option) *Server {
    s := &Server{host: host, port: 8080, timeout: 30 * time.Second}
    for _, opt := range opts { opt(s) }
    return s
}

srv := NewServer("localhost",
    WithPort(9090),
    WithTimeout(60*time.Second),
)
```

> ***Interview tip:*** *"This pattern is in gRPC, zap, and most modern Go libraries. Knowing it signals production experience."*

Q84. What is the builder pattern in Go?

Pattern: Builder

Difficulty: Medium

Chain methods to construct complex objects. Each method returns the builder for chaining. Call a final `Build()` to get the result.

```
type QueryBuilder struct {
    table  string
    wheres []string
    limit  int
    order  string
}

func (qb *QueryBuilder) From(table string) *QueryBuilder {
    qb.table = table; return qb
}
func (qb *QueryBuilder) Where(cond string) *QueryBuilder {
    qb.wheres = append(qb.wheres, cond); return qb
```

```
}
func (qb *QueryBuilder) Limit(n int) *QueryBuilder {
    qb.limit = n; return qb
}
func (qb *QueryBuilder) Build() string {
    q := "SELECT * FROM " + qb.table
    if len(qb.wheres) > 0 {
        q += " WHERE " + strings.Join(qb.wheres, " AND ")
    }
    if qb.limit > 0 { q += fmt.Sprintf(" LIMIT %d", qb.limit) }
    return q
}

query := (&QueryBuilder{}).From("users").
    Where("age > 18").Where("active = true").Limit(10).Build()
```

Interview tip: *"Builder is useful when construction requires many steps or optional parts. Prefer functional options for simpler cases."*

Q85. What is the decorator pattern in Go?

Pattern: Decorator | **Difficulty: Medium**

Wrap a function or interface implementation with additional behaviour. Same interface, enhanced functionality — the basis for HTTP middleware.

```
type HandlerFunc func(ctx context.Context, req Request) (Response, error)

func WithLogging(h HandlerFunc, logger *log.Logger) HandlerFunc {
    return func(ctx context.Context, req Request) (Response, error) {
        start := time.Now()
        resp, err := h(ctx, req)
        logger.Printf("req=%v dur=%v err=%v", req, time.Since(start), err)
        return resp, err
    }
}

func WithMetrics(h HandlerFunc, metrics *Metrics) HandlerFunc {
    return func(ctx context.Context, req Request) (Response, error) {
        resp, err := h(ctx, req)
        metrics.Record(req, err)
        return resp, err
    }
}

handler := WithMetrics(WithLogging(businessLogic, logger), metrics)
```

Interview tip: *"Decorators are composable — each adds one concern. Prefer over subclassing (which Go doesn't have anyway)."*

Q86. What is the strategy pattern in Go?

Pattern: Strategy | **Difficulty: Medium**

Define a family of algorithms, encapsulate each one, and make them interchangeable via an interface.

```
type SortStrategy interface {
    Sort(data []int) []int
}
type BubbleSort struct{}
```

```
func (b BubbleSort) Sort(data []int) []int {
    // bubble sort implementation
    return data
}

type QuickSort struct{}
func (q QuickSort) Sort(data []int) []int {
    // quicksort implementation
    return data
}

type Sorter struct{ strategy SortStrategy }
func (s *Sorter) SetStrategy(st SortStrategy) { s.strategy = st }
func (s *Sorter) Sort(data []int) []int       { return s.strategy.Sort(data) }

sorter := &Sorter{strategy: QuickSort{}}
result := sorter.Sort([]int{5, 3, 1, 4, 2})

// Switch strategy at runtime
sorter.SetStrategy(BubbleSort{})
```

Interview tip: *"In Go, the strategy pattern is naturally expressed via interfaces. Almost every interface you define is a strategy."*

Q87. What is the circuit breaker pattern?

Pattern: Resilience

Difficulty: Hard

Wraps external calls. After N failures, opens the circuit (fails fast). After a timeout, goes half-open (tries one request). If it succeeds, closes the circuit.

```
type State int
const (Closed State = iota; Open; HalfOpen)

type CircuitBreaker struct {
    mu           sync.Mutex
    state        State
    failures     int
    maxFailures  int
    timeout      time.Duration
    lastFailure  time.Time
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    cb.mu.Lock()
    switch cb.state {
    case Open:
        if time.Since(cb.lastFailure) > cb.timeout {
            cb.state = HalfOpen
        } else {
            cb.mu.Unlock()
            return errors.New("circuit open")
        }
    }
    cb.mu.Unlock()

    err := fn()

    cb.mu.Lock()
```

```
    defer cb.mu.Unlock()
    if err != nil {
        cb.failures++
        cb.lastFailure = time.Now()
        if cb.failures >= cb.maxFailures { cb.state = Open }
        return err
    }
    cb.failures = 0
    cb.state = Closed
    return nil
}
```

> ***Interview tip:*** *"`github.com/sony/gobreaker` is a production-ready implementation. Know the concept; don't reinvent in production."*

Q88. What is the repository pattern in Go?

Pattern: Data access | **Difficulty: Medium**

Abstracts data access behind an interface. Business logic depends on the interface, not the DB. Enables swapping implementations (PostgreSQL, in-memory, mock).

```
type User struct {
    ID    int
    Name  string
    Email string
}

type UserRepository interface {
    GetByID(ctx context.Context, id int) (*User, error)
    Save(ctx context.Context, u *User) error
    Delete(ctx context.Context, id int) error
    ListByEmail(ctx context.Context, email string) ([]*User, error)
}

// PostgreSQL implementation
type PostgresUserRepo struct{ db *sql.DB }

func (r *PostgresUserRepo) GetByID(ctx context.Context, id int) (*User, error) {
    var u User
    err := r.db.QueryRowContext(ctx,
        "SELECT id, name, email FROM users WHERE id=$1", id).
        Scan(&u.ID, &u.Name, &u.Email)
    if err != nil { return nil, fmt.Errorf("GetByID: %w", err) }
    return &u, nil
}

// In-memory for tests
type InMemoryUserRepo struct {
    mu    sync.RWMutex
    users map[int]*User
}
```

> ***Interview tip:*** *"The repository pattern is standard in Go microservices. It enables unit testing business logic without a real database."*

Q89. What is the command pattern in Go?

Pattern: Command | **Difficulty: Medium**

Encapsulate a request as an object. Supports undo/redo, queuing, and logging of operations.

```
type Command interface {
    Execute() error
    Undo() error
}

type AddItemCommand struct {
    cart *ShoppingCart
    item Item
}

func (c *AddItemCommand) Execute() error {
    return c.cart.Add(c.item)
}

func (c *AddItemCommand) Undo() error {
    return c.cart.Remove(c.item)
}

type CommandHistory struct{ history []Command }

func (ch *CommandHistory) Execute(cmd Command) error {
    if err := cmd.Execute(); err != nil { return err }
    ch.history = append(ch.history, cmd)
    return nil
}

func (ch *CommandHistory) Undo() error {
    if len(ch.history) == 0 { return errors.New("nothing to undo") }
    last := ch.history[len(ch.history)-1]
    ch.history = ch.history[:len(ch.history)-1]
    return last.Undo()
}
```

> ***Interview tip:*** *"Command pattern shines in editors, CLIs, and anything needing undo/redo. The interface makes every operation first-class."*

Q90. What is the adapter pattern in Go?

Pattern: Adapter | **Difficulty: Easy**

Convert an incompatible interface into the interface the client expects. Wraps an existing type.

```
// External library uses old interface
type OldLogger struct{}
func (l *OldLogger) Log(msg string) { fmt.Println("[OLD]", msg) }

// Our system expects new interface
type Logger interface {
    Info(msg string)
    Error(msg string)
}

// Adapter
type LoggerAdapter struct{ old *OldLogger }

func (a *LoggerAdapter) Info(msg string)  { a.old.Log("[INFO] " + msg) }
func (a *LoggerAdapter) Error(msg string) { a.old.Log("[ERROR] " + msg) }
```

```
// Usage
var l Logger = &LoggerAdapter{old: &OldLogger{}}
l.Info("service started")
```

> ***Interview tip:*** *"Adapters are common when integrating third-party libraries. They're the glue between 'what we have' and 'what we need'."*

Q91. How do you implement graceful HTTP server shutdown in Go?

Pattern: Graceful shutdown

Difficulty: Hard

Use `server.Shutdown(ctx)` to stop accepting new connections and wait for active requests to complete.

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", handler)

    srv := &http.Server{
        Addr:         ":8080",
        Handler:      mux,
        ReadTimeout:  15 * time.Second,
        WriteTimeout: 15 * time.Second,
        IdleTimeout:  60 * time.Second,
    }

    go func() {
        if err := srv.ListenAndServe(); err != http.ErrServerClosed {
            log.Fatalf("ListenAndServe: %v", err)
        }
    }()

    // Wait for interrupt signal
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    log.Println("shutting down server...")
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

    if err := srv.Shutdown(ctx); err != nil {
        log.Fatalf("Server forced to shutdown: %v", err)
    }
    log.Println("server exited")
}
```

> ***Interview tip:*** *"The 30-second timeout gives in-flight requests time to complete. In Kubernetes, set this to match your `terminationGracePeriodSeconds`."*

Q92. What is dependency injection in Go?

Pattern: Dependency injection

Difficulty: Medium

Pass dependencies explicitly through constructors or function parameters. No framework needed — Go's interfaces make DI natural.

```
type EmailSender interface {
    Send(to, subject, body string) error
```

```
}

type UserService struct {
    repo   UserRepository
    email  EmailSender
    logger *log.Logger
}

func NewUserService(
    repo UserRepository,
    email EmailSender,
    logger *log.Logger,
) *UserService {
    return &UserService{repo: repo, email: email, logger: logger}
}

func (s *UserService) Register(ctx context.Context, user *User) error {
    if err := s.repo.Save(ctx, user); err != nil {
        return fmt.Errorf("register: %w", err)
    }
    return s.email.Send(user.Email, "Welcome!", "Thanks for signing up")
}

// Wire up in main()
svc := NewUserService(
    &PostgresUserRepo{db: db},
    &SMTPSender{host: "smtp.example.com"},
    log.Default(),
)
```

> ***Interview tip:*** *"Prefer manual DI over frameworks like Wire for small/medium services. For large codebases, Google's Wire generates boilerplate."*

Miscellaneous & Advanced

Q93. What are Go modules and how does go.mod work?

Pattern: Module system

Difficulty: Easy

Go modules (introduced in Go 1.11) manage dependencies. `go.mod` defines the module path, Go version, and dependencies. `go.sum` stores checksums for verification.

```
// go.mod
module github.com/yourname/myapp

go 1.21

require (
    github.com/gin-gonic/gin v1.9.1
    golang.org/x/sync v0.5.0
)

require (
    // indirect dependencies
    github.com/bytedance/sonic v1.10.2 // indirect
```

```
)
go mod init github.com/yourname/myapp
go get github.com/gin-gonic/gin@v1.9.1
go mod tidy          # remove unused, add missing
go mod vendor        # copy deps to ./vendor
go list -m all       # list all modules
```

> ***Interview tip:*** *"Always run `go mod tidy` before committing. It ensures go.mod and go.sum are clean and consistent."*

Q94. What is the difference between Go's defer, panic, and recover?

Pattern: Defer/panic/recover

Difficulty: Medium

`defer` runs a function when the surrounding function returns, in LIFO order. `panic` unwinds the stack, calling deferred functions. `recover` catches the panic if called inside a deferred function.

```
func safeOperation() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("panic recovered: %v", r)
        }
    }()

    riskyOperation() // might panic
    return nil
}

// LIFO order
func main() {
    defer fmt.Println("third")  // runs 3rd (last deferred, first executed)
    defer fmt.Println("second")
    defer fmt.Println("first")  // runs 1st
    fmt.Println("main body")
}
// Output: main body, first, second, third
```

> ***Interview tip:*** *"Defer is evaluated immediately but called later. `defer fmt.Println(x)` captures x's value at the defer statement."*

Q95. What is reflection in Go and when do you use it?

Pattern: Reflection

Difficulty: Hard

`reflect` package lets you inspect and manipulate types at runtime. Used in JSON marshalling, ORMs, and dependency injection frameworks. Slow — avoid in hot paths.

```
func inspect(v any) {
    t := reflect.TypeOf(v)
    val := reflect.ValueOf(v)

    fmt.Printf("Type: %s\n", t.Name())
    fmt.Printf("Kind: %s\n", t.Kind())

    if t.Kind() == reflect.Struct {
        for i := 0; i < t.NumField(); i++ {
            field := t.Field(i)
            value := val.Field(i)
            fmt.Printf("Field: %s = %v (tag: %s)\n",
```

```
                field.Name, value, field.Tag.Get("json"))
        }
    }
}

type User struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}

inspect(User{"Alice", 30})
```

> ***Interview tip:*** *"Reflection is used by `encoding/json`, `database/sql`, and testing frameworks. In application code, prefer generics over reflection."*

Q96. How does Go handle JSON marshalling and unmarshalling?

Pattern: JSON | **Difficulty: Easy**

`encoding/json` uses struct tags to map fields. `Marshaler`/`Unmarshaler` interfaces allow custom behaviour. `json.Decoder` for streaming, `json.Unmarshal` for in-memory.

```
type User struct {
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    CreatedAt time.Time `json:"created_at"`
    Password  string    `json:"-"`            // never serialised
    Age       int       `json:"age,omitempty"` // omitted if zero
}

// Marshal
u := User{Name: "Alice", Email: "alice@example.com"}
data, err := json.Marshal(u)

// Unmarshal
var u2 User
err = json.Unmarshal(data, &u2)

// Streaming decoder (for large responses)
dec := json.NewDecoder(resp.Body)
for dec.More() {
    var u User
    if err := dec.Decode(&u); err != nil { break }
    process(u)
}
```

> ***Interview tip:*** *"Use `json.Decoder` for HTTP response bodies and files — it doesn't load everything into memory at once."*

Q97. What is CGo and when would you use it?

Pattern: CGo | **Difficulty: Hard**

CGo enables calling C code from Go. Used for system libraries, performance-critical native code, or interfacing with C SDKs. Has significant overhead per call.

```
/*
#include <stdio.h>
#include <stdlib.h>

void hello() {
```

```
    printf("Hello from C!\n");
}
*/
import "C"
import "unsafe"

func main() {
    C.hello() // call C function

    // C string handling
    cs := C.CString("Go string to C")
    defer C.free(unsafe.Pointer(cs)) // must free!
    C.puts(cs)
}
CGO_ENABLED=0 go build  # disable CGo for static binary
```

> ***Interview tip:*** *"CGo has ~100ns overhead per call. Don't use it for tight loops. Prefer pure Go libraries when available. CGo complicates cross-compilation."*

Q98. How do you implement a custom sort in Go?

Pattern: Sorting | **Difficulty: Easy**

Implement `sort.Interface` (Len, Less, Swap) for custom types, or use `sort.Slice` / `sort.SliceStable` for closures.

```
type Person struct{ Name string; Age int }

// sort.Slice — easiest
people := []Person{{"Alice", 30}, {"Bob", 25}, {"Charlie", 35}}
sort.Slice(people, func(i, j int) bool {
    return people[i].Age < people[j].Age
})

// sort.Interface — for reusable sorts
type ByName []Person
func (b ByName) Len() int           { return len(b) }
func (b ByName) Less(i, j int) bool { return b[i].Name < b[j].Name }
func (b ByName) Swap(i, j int)      { b[i], b[j] = b[j], b[i] }
sort.Sort(ByName(people))

// Go 1.21+ — slices package
import "slices"
slices.SortFunc(people, func(a, b Person) int {
    return strings.Compare(a.Name, b.Name)
})
```

> ***Interview tip:*** *"In Go 1.21+, prefer `slices.Sort` and `slices.SortFunc` — they're faster (pdqsort) and more ergonomic."*

Q99. What are some common Go anti-patterns to avoid?

Pattern: Anti-patterns | **Difficulty: Medium**

```
// 1. Naked return (confusing in long functions)
func bad() (x int) { x = 5; return } // avoid

// 2. Ignoring errors
data, _ := os.ReadFile("file.txt") // NEVER in production
```

```
// 3. Goroutine without a way to stop
go func() { for { doWork() } }() // where does this end?

// 4. Using interface{} where generics fit better
func sumBad(items []interface{}) float64 { /* type switches */ }

// 5. init() side effects
func init() { connectToDB() } // hard to test, hidden dependency

// 6. Global mutable state
var globalCache = map[string]any{} // not safe for concurrent use

// 7. Premature sync.Pool usage
// Don't use Pool for objects that escape frequently

// 8. Context stored in struct
type S struct{ ctx context.Context } // WRONG

// 9. Unbounded goroutine spawn
for _, item := range millionItems {
    go process(item) // spawns 1M goroutines!
}
// Fix: use worker pool

// 10. String + concatenation in loop
s := ""
for _, word := range words {
    s += word // O(n²) — use strings.Builder
}
```

> ***Interview tip:*** *"Knowing anti-patterns shows maturity. Interviewers appreciate when you proactively call out what NOT to do."*

Q100. How do you design a Go service for production readiness?

Pattern: Production readiness

Difficulty: Hard

A production-ready Go service requires: structured logging, metrics, distributed tracing, health checks, graceful shutdown, circuit breakers, and proper configuration management.

```
func main() {
    // 1. Configuration
    cfg := config.Load() // from env / Vault

    // 2. Structured logging
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    // 3. Metrics (Prometheus)
    reg := prometheus.NewRegistry()
    http.Handle("/metrics", promhttp.HandlerFor(reg, promhttp.HandlerOpts{}))

    // 4. Tracing (OpenTelemetry)
    tp := initTracer(cfg.JaegerEndpoint)
    defer tp.Shutdown(context.Background())

    // 5. Database with connection pool
    db, _ := sql.Open("postgres", cfg.DSN)
    db.SetMaxOpenConns(25)
```

```
    db.SetMaxIdleConns(10)
    db.SetConnMaxLifetime(5 * time.Minute)

    // 6. Service with dependencies
    repo := &PostgresUserRepo{db: db}
    svc := NewUserService(repo, logger)

    // 7. HTTP server with timeouts
    srv := &http.Server{
        Addr:         ":" + cfg.Port,
        Handler:      router(svc, logger),
        ReadTimeout:  15 * time.Second,
        WriteTimeout: 15 * time.Second,
        IdleTimeout:  60 * time.Second,
    }

    // 8. Health check
    http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
        if err := db.PingContext(r.Context()); err != nil {
            w.WriteHeader(503)
            return
        }
        w.WriteHeader(200)
    })

    // 9. Graceful shutdown
    go srv.ListenAndServe()
    waitForShutdown(srv, 30*time.Second)
}
```

> ***Interview tip:*** *"Production readiness is the difference between a junior and a senior engineer. Mentioning observability, timeouts, and graceful shutdown unprompted will impress any interviewer."*

Quick Reference: Complexity Cheat Sheet

Data Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Slice (append)	O(1)	O(n)	O(1) amortised	O(n)
Map	O(1) avg	O(1) avg	O(1) avg	O(1) avg
Linked List	O(n)	O(n)	O(1)	O(1)
Heap	O(n)	O(n)	O(log n)	O(log n)
BST	O(log n) avg	O(log n) avg	O(log n) avg	O(log n) avg

Quick Reference: Go Concurrency Primitives

Primitive	Use Case	Thread-Safe
`goroutine`	Concurrent execution	N/A

`chan T`	Communication	Yes
`sync.Mutex`	Exclusive access	Yes
`sync.RWMutex`	Many readers, one writer	Yes
`sync.WaitGroup`	Wait for goroutines	Yes
`sync.Once`	One-time initialisation	Yes
`sync.Pool`	Object reuse	Yes
`sync.Map`	Concurrent map	Yes
`atomic.*`	Lock-free counters	Yes
`context.Context`	Cancellation/deadlines	Yes

Good luck with your SDE2 interviews! ■

Star this repo if it helped you — and contribute questions you encountered in real interviews.